

# 透過 What-If Tool 分析部署至 AI 平台上的財務機器學習模型

來源：<https://codelabs.developers.google.com/codelabs/xgb-wit-caip?hl=zh-tw>

步驟數：8

在這個研究室中，您將瞭解如何在金融資料集中訓練 XGBoost 模型、將模型部署至 Cloud AI 平台，並使用 What-if Tool 加以分析。

---

# 目錄

1. [總覽](#)
2. [XGBoost 快速入門](#)
3. [設定環境](#)
4. [下載及處理資料](#)
5. [建構、訓練及評估 XGBoost 模型](#)
6. [將模型部署至 Cloud AI Platform](#)
7. [使用 What-If Tool 解讀模型](#)
8. [清除所用資源](#)

步驟 1 · 約 1 分鐘

## 1. 總覽

在本實驗室中，您將使用 [What-if Tool](#) 分析以財務資料訓練，並部署在 Cloud AI Platform 上的 [XGBoost](#) 模型。

## 課程內容

內容如下：

- 在 AI Platform Notebooks 中，根據公開的抵押資料集訓練 XGBoost 模型
- 將 XGBoost 模型部署至 AI Platform
- 使用 What-If Tool 分析模型

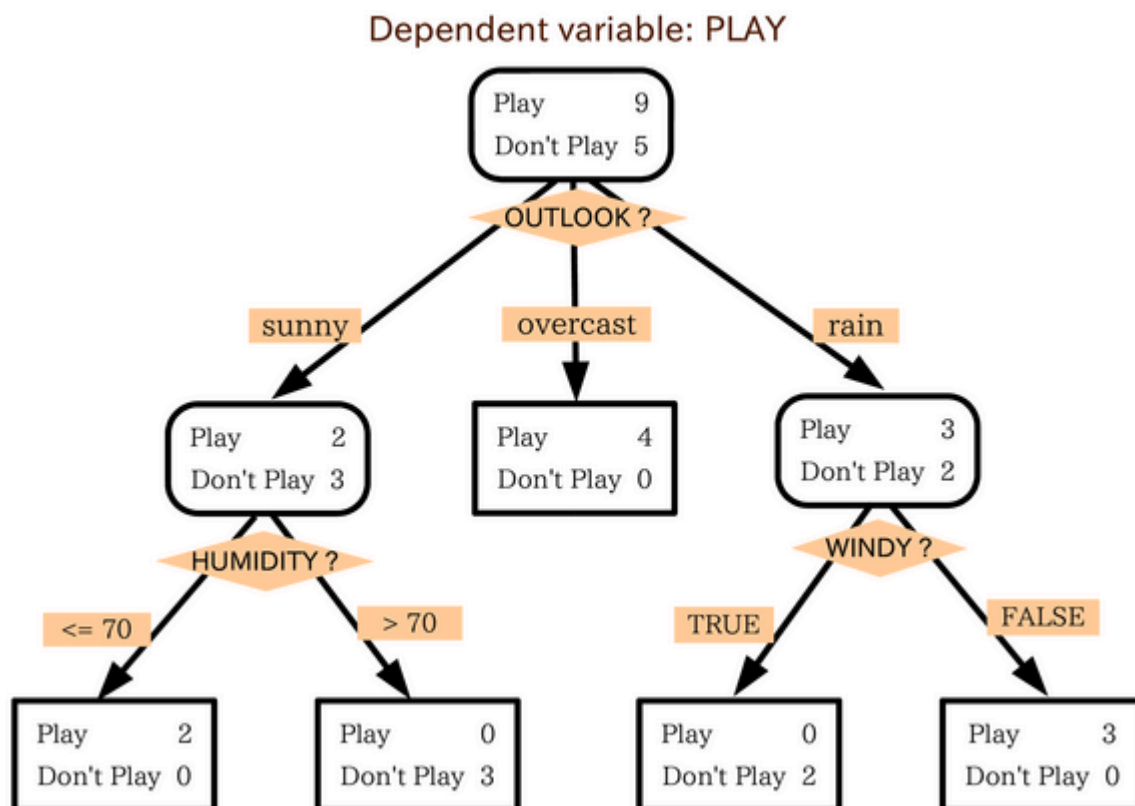
在 Google Cloud 上執行這個實驗室的總費用約為 **\$1 美元**。

---

## 2. XGBoost 快速入門

**XGBoost** 是一種機器學習框架，可使用**決策樹**和**梯度提升**來建構預測模型。這項技術會根據樹狀結構中不同葉節點的相關分數，將多個決策樹集合在一起。

下圖是簡單決策樹模型的**視覺化**呈現，可根據天氣預報評估是否應進行體育賽事：



為什麼要使用 **XGBoost** 建立這個模型？傳統類神經網路在圖像和文字等非結構化資料方面表現最佳，而決策樹在結構化資料方面通常表現極佳，例如我們將在本程式碼研究室中使用的房貸資料集。

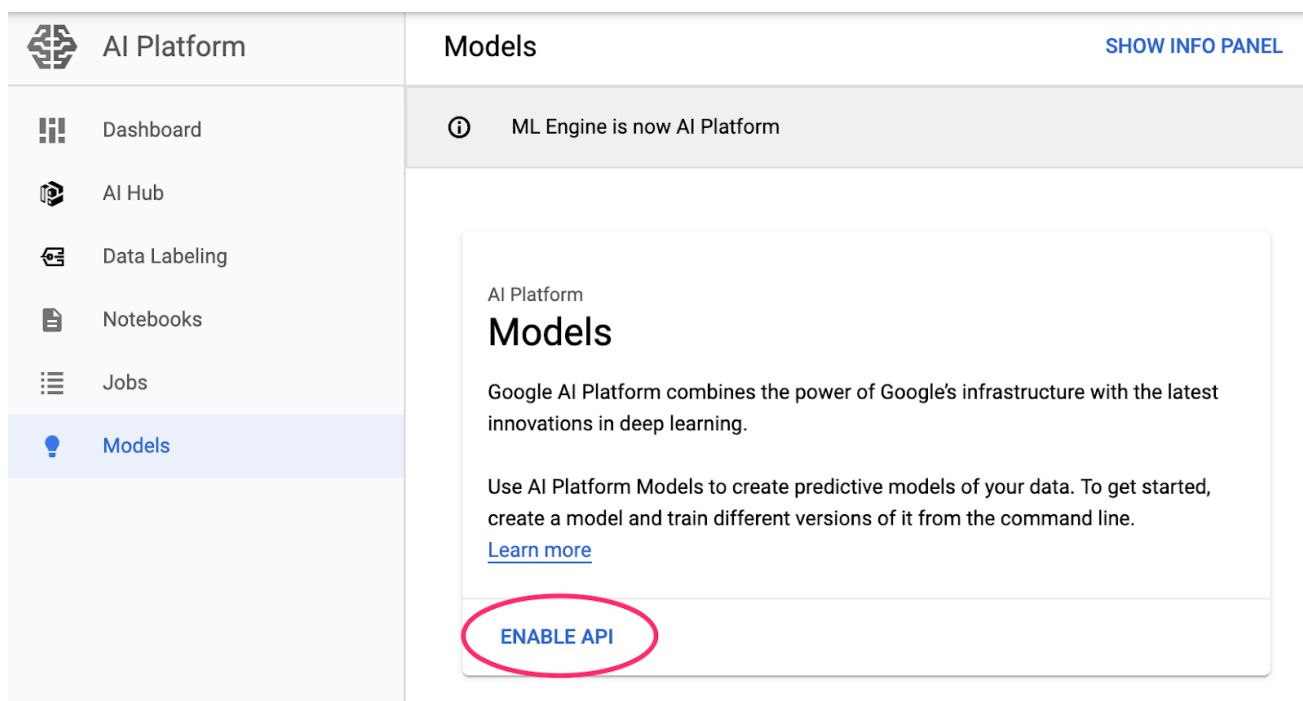
步驟 3 · 約 10 分鐘

### 3. 設定環境

您必須擁有已啟用計費功能的 Google Cloud Platform 專案，才能執行這項程式碼研究室。如要建立專案，請按照[這裡](#)的操作說明進行。

#### 步驟 1：啟用 Cloud AI Platform Models API

前往 Cloud 控制台的 [AI Platform Models 區段](#)，如果尚未啟用，請點選「啟用」。



#### 步驟 2：啟用 Compute Engine API

前往「Compute Engine」，然後選取「啟用」（如果尚未啟用）。您需要這項資訊才能建立筆記本執行個體。

#### 步驟 3：建立 AI Platform Notebooks 執行個體

前往 Cloud Console 的 [AI Platform Notebooks 專區](#)，然後點選「建立執行個體」。然後選取「不含 GPU」的「最新 TF 企業版 2.x」執行個體類型：

##### Customize instance

###### R 3.6

Includes scikit-learn, pandas, NLTK and more

###### Python 2 and 3

Includes scikit-learn, pandas and more

###### CUDA Toolkit 10.1

Optimized for NVIDIA GPUs

###### TensorFlow Enterprise 1.15

Includes Keras, scikit-learn, pandas, NLTK and more

###### TensorFlow Enterprise 2.1

Includes Keras, scikit-learn, pandas, NLTK and more

使用預設選項，然後按一下「建立」。建立執行個體後，請選取「Open JupyterLab」：



sara-xgb-mortgage-  
codelab

[OPEN JUPYTERLAB](#)

us-  
west1-b

TensorFlow:1.15

4 vCPUs, 15 GB  
RAM

None

Compute Engine default  
service account

## 步驟 4：安裝 XGBoost

開啟 JupyterLab 執行個體後，您需要新增 XGBoost 套件。

如要這麼做，請從啟動器選取「終端機」：



Notebook



Python 3



Python 2



Console



Python 3



Python 2

Other



Terminal



Text File



Tensorboard

然後執行下列指令，安裝 Cloud AI Platform 支援的最新版 XGBoost：

```
pip3 install xgboost==0.90
```

完成後，請從啟動器開啟 Python 3 Notebook 執行個體。您可以在筆記本中開始使用！

## 步驟 5：匯入 Python 套件

在本程式碼研究室的其餘部分，請從 Jupyter 筆記本執行所有程式碼片段。

在筆記本的第一個儲存格中，新增下列匯入內容並執行儲存格。如要執行，請按下頂端選單中的向右箭頭按鈕，或按下 Command-Enter 鍵：

```
import pandas as pd
import xgboost as xgb
import numpy as np
import collections
import witwidget

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.utils import shuffle
from witwidget.notebook.visualization import WitWidget, WitConfigBuilder
```

---

步驟 4 · 約 5 分鐘

## 4. 下載及處理資料

我們將使用 [ffiec.gov](https://ffiec.gov) 的房貸資料集訓練 XGBoost 模型。我們已對原始資料集進行一些前置處理，並建立較小的版本供您訓練模型。模型會預測特定房貸申請是否會獲得核准。

### 步驟 1：下載預先處理的資料集

我們已在 Google Cloud Storage 中提供資料集版本。如要下載，請在 Jupyter 筆記本中執行下列 `gsutil` 指令：

```
!gsutil cp 'gs://mortgage_dataset_files/mortgage-small.csv' .
```

### 步驟 2：使用 Pandas 讀取資料集

建立 Pandas DataFrame 前，我們會先建立每個資料欄資料類型的字典，讓 Pandas 正確讀取資料集：

```
COLUMN_NAMES = collections.OrderedDict({
    'as_of_year': np.int16,
    'agency_code': 'category',
    'loan_type': 'category',
    'property_type': 'category',
    'loan_purpose': 'category',
    'occupancy': np.int8,
    'loan_amt_thousands': np.float64,
    'preapproval': 'category',
    'county_code': np.float64,
    'applicant_income_thousands': np.float64,
    'purchaser_type': 'category',
    'hoepa_status': 'category',
    'lien_status': 'category',
    'population': np.float64,
    'ffiec_median_fam_income': np.float64,
    'tract_to_msa_income_pct': np.float64,
    'num_owner_occupied_units': np.float64,
    'num_1_to_4_family_units': np.float64,
    'approved': np.int8
})
```

接著，我們將建立 DataFrame，並傳遞上方指定的資料類型。如果原始資料集是以特定方式排序，請務必隨機排序資料。我們使用名為 `shuffle` 的 `sklearn` 公用程式執行這項操作，該公用程式已在第一個儲存格中匯入：

```
data = pd.read_csv(
    'mortgage-small.csv',
    index_col=False,
    dtype=COLUMN_NAMES
)
data = data.dropna()
data = shuffle(data, random_state=2)
data.head()
```



`data.head()` 可讓我們在 Pandas 中預覽資料集的前五列。執行上述儲存格後，您應該會看到類似下列的內容：

|        | as_of_year | agency_code                                       | loan_type                                         | property_type                                     | loan_purpose  | occupancy | loan_amt_thousands | preapproval    | county_code | applicant_income_thousands | purchaser_type                                    |
|--------|------------|---------------------------------------------------|---------------------------------------------------|---------------------------------------------------|---------------|-----------|--------------------|----------------|-------------|----------------------------|---------------------------------------------------|
| 310650 | 2016       | Consumer Financial Protection Bureau (CFPB)       | Conventional (any loan other than FHA, VA, FSA... | One to four-family (other than manufactured ho... | Refinancing   | 1         | 110.0              | Not applicable | 119.0       | 55.0                       | Freddie Mac (FHLMC)                               |
| 630129 | 2016       | Department of Housing and Urban Development (HUD) | Conventional (any loan other than FHA, VA, FSA... | One to four-family (other than manufactured ho... | Home purchase | 1         | 480.0              | Not applicable | 33.0        | 270.0                      | Loan was not originated or was not sold in cal... |
| 715484 | 2016       | Federal Deposit Insurance Corporation (FDIC)      | Conventional (any loan other than FHA, VA, FSA... | One to four-family (other than manufactured ho... | Refinancing   | 2         | 240.0              | Not applicable | 59.0        | 96.0                       | Commercial bank, savings bank or savings assoc... |
| 887708 | 2016       | Office of the Comptroller of the Currency (OCC)   | Conventional (any loan other than FHA, VA, FSA... | One to four-family (other than manufactured ho... | Refinancing   | 1         | 76.0               | Not applicable | 65.0        | 85.0                       | Loan was not originated or was not sold in cal... |
| 719598 | 2016       | National Credit Union Administration (NCUA)       | Conventional (any loan other than FHA, VA, FSA... | One to four-family (other than manufactured ho... | Refinancing   | 1         | 100.0              | Not applicable | 127.0       | 70.0                       | Loan was not originated or was not sold in cal... |

這些是我們用來訓練模型的特徵。捲動至最後，您會看到最後一欄 `approved`，這是我們要預測的內容。值為 `1` 表示特定應用程式已獲准，值為 `0` 則表示遭拒。

如要查看資料集中核准 / 拒絕值的分布情形，並建立標籤的 NumPy 陣列，請執行下列指令：

```
# Class labels - 0: denied, 1: approved
print(data['approved'].value_counts())

labels = data['approved'].values
data = data.drop(columns=['approved'])
```

資料集中約有 66% 的核准應用程式。

**不平衡資料集提示：**如果模型準確率接近資料集中核准 / 拒絕項目的確切比例 (在本例中為 66%)，表示模型可能隨機猜測。如果準確率遠高於 66%，表示模型正在學習。

## 步驟 3：為類別值建立虛擬資料欄

這個資料集包含類別和數值，但 XGBoost 要求所有特徵都必須是數值。我們將利用 Pandas `get_dummies` 函式，而非使用單一熱編碼來表示類別值，以供 XGBoost 模型使用。

`get_dummies` 會採用具有多個可能值的資料欄，並轉換為一系列只有 0 和 1 的資料欄。舉例來說，假設我們有一個「color」資料欄，可能的值為「blue」和「red」，`get_dummies` 會將此轉換為 2 個名為「color\_blue」和「color\_red」的資料欄，並包含所有布林值 0 和 1。

如要為類別特徵建立虛擬資料欄，請執行下列程式碼：

```
dummy_columns = list(data.dtypes[data.dtypes == 'category'].index)
data = pd.get_dummies(data, columns=dummy_columns)

data.head()
```

這次預覽資料時，您會看到單一特徵 (如下圖所示的 `purchaser_type`) 分割成多個資料欄：

| <b>purchaser_type_Life<br/>insurance company,<br/>credit union,<br/>mortgage bank, or<br/>finance company</b> | <b>purchaser_type_Loan<br/>was not originated or<br/>was not sold in<br/>calendar year<br/>covered by register</b> | <b>purchaser_type_Other<br/>type of purchaser</b> | <b>purchaser_type_Private<br/>securitization</b> |
|---------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|---------------------------------------------------|--------------------------------------------------|
| 0                                                                                                             | 0                                                                                                                  | 0                                                 | 0                                                |
| 0                                                                                                             | 1                                                                                                                  | 0                                                 | 0                                                |
| 0                                                                                                             | 0                                                                                                                  | 0                                                 | 0                                                |
| 0                                                                                                             | 1                                                                                                                  | 0                                                 | 0                                                |
| 0                                                                                                             | 1                                                                                                                  | 0                                                 | 0                                                |

## 步驟 4：將資料拆分為訓練集和測試集

機器學習的重要概念是訓練 / 測試分割。我們會使用大部分資料訓練模型，並保留其餘資料，用於測試模型是否能處理從未見過的資料。

在筆記本中新增下列程式碼，這段程式碼會使用 Scikit Learn 函式 `train_test_split` 分割資料：

```
x,y = data,labels
x_train,x_test,y_train,y_test = train_test_split(x,y)
```

現在，您可以建構及訓練模型了！

---

步驟 5 · 約 5 分鐘

## 5. 建構、訓練及評估 XGBoost 模型

### 步驟 1：定義及訓練 XGBoost 模型

在 XGBoost 中建立模型非常簡單。我們會使用 `XGBClassifier` 類別建立模型，只需要為特定分類工作傳遞正確的 `objective` 參數即可。在本範例中，我們使用 `reg:logistic`，因為我們有二元分類問題，且希望模型在 (0,1) 範圍內輸出單一值：0 代表未核准，1 代表已核准。

下列程式碼會建立 XGBoost 模型：

```
model = xgb.XGBClassifier(  
    objective='reg:logistic'  
)
```

您只需呼叫 `fit()` 方法，並傳遞訓練資料和標籤，即可使用一行程式碼訓練模型。

```
model.fit(x_train, y_train)
```

**注意：**訓練模型需要幾分鐘。

### 步驟 2：評估模型準確率

現在可以使用 `predict()` 函式，透過訓練好的模型對測試資料產生預測結果。

接著，我們會使用 Scikit Learn 的 `accuracy_score` 函式，根據模型在測試資料上的表現計算準確率。我們會將真值連同模型為測試集中每個樣本預測的值傳遞給這個函式：

```
y_pred = model.predict(x_test)  
acc = accuracy_score(y_test, y_pred.round())  
print(acc, '\n')
```

您應該會看到準確率約為 **87%**，但由於機器學習一律含有隨機性元素，因此您的準確率會略有不同。

### 步驟 3：儲存模型

如要部署模型，請執行下列程式碼，將模型儲存至本機檔案：

```
model.save_model('model.bst')
```

---

步驟 6 · 約 10 分鐘

## 6. 將模型部署至 Cloud AI Platform

我們已在本機運作模型，但如果能從任何位置 (不只是這個筆記本！) 預測模型，那就太好了。在這個步驟中，我們會將其部署至雲端。

### 步驟 1：為模型建立 Cloud Storage bucket

首先，請定義一些環境變數，我們會在程式碼研究室的其餘部分使用這些變數。請在下方填入 Google Cloud 雲端專案名稱、要建立的 Cloud Storage 值區名稱 (必須是全域不重複的名稱)，以及模型第一個版本的版本名稱：

```
# Update these to your own GCP project, model, and version names
GCP_PROJECT = 'your-gcp-project'
MODEL_BUCKET = 'gs://storage_bucket_name'
VERSION_NAME = 'v1'
MODEL_NAME = 'xgb_mortgage'
```

現在我們準備建立儲存值區，以儲存 XGBoost 模型檔案。部署時，我們會將 Cloud AI Platform 指向這個檔案。

在筆記本中執行下列 `gsutil` 指令，建立 bucket：

```
!gsutil mb $MODEL_BUCKET
```

### 步驟 2：將模型檔案複製到 Cloud Storage

接著，我們會將 XGBoost 儲存的模型檔案複製到 Cloud Storage。執行下列 `gsutil` 指令：

```
!gsutil cp ./model.bst $MODEL_BUCKET
```

前往 Cloud 控制台中的儲存空間瀏覽器，確認檔案已複製完成：

[Buckets](#) / mortgage\_model\_bucket

| <input type="checkbox"/> | Name                                                                                          | Size     | Type                     | Storage class  | Last modified             | Public access <a href="#">?</a> | Encryption <a href="#">?</a> |
|--------------------------|-----------------------------------------------------------------------------------------------|----------|--------------------------|----------------|---------------------------|---------------------------------|------------------------------|
| <input type="checkbox"/> |  model.bst | 48.63 KB | application/octet-stream | Multi-Regional | 7/23/19, 5:18:57 PM UTC-7 | Not public                      | Google-managed key           |

### 步驟 3：建立及部署模型

我們即將部署模型！下列 `ai-platform gcloud` 指令會在專案中建立新模型。我們將這個帳戶稱為「`xgb_mortgage`」：

```
!gcloud ai-platform models create $MODEL_NAME --region='global'
```

現在可以部署模型了。我們可以使用下列 `gcloud` 指令執行這項操作：

```
!gcloud ai-platform versions create $VERSION_NAME \
--model=$MODEL_NAME \
--framework='XGBOOST' \
--runtime-version=2.1 \
--origin=$MODEL_BUCKET \
--python-version=3.7 \
--project=$GCP_PROJECT \
--region='global'
```

執行這項作業時，請檢查 AI Platform 控制台的[模型部分](#)。您應該會看到新版本正在部署：

| <input type="checkbox"/> |  | Name            | Create time                       | Last used | Evaluation | Labels |
|--------------------------|-----------------------------------------------------------------------------------|-----------------|-----------------------------------|-----------|------------|--------|
| <input type="checkbox"/> |  | awesome_version | Jul 16,<br>2019,<br>9:32:54<br>AM |           | N/A        |        |

部署作業完成後，載入旋轉圖示會變成綠色勾號。部署作業應會在 **2 到 3 分鐘**內完成。

## 步驟 4：測試已部署的模型

如要確認已部署的模型是否正常運作，請使用 `gcloud` 進行預測測試。首先，請儲存 JSON 檔案，並使用測試集中的第一個範例：

```
%%writefile predictions.json
[2016.0, 1.0, 346.0, 27.0, 211.0, 4530.0, 86700.0, 132.13, 1289.0, 1408.0, 0.0, 0.0, 0.0,
0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0, 0.0, 1.0, 0.0,
0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.0, 1.0, 0.0]
```

執行下列程式碼來測試模型：

```
prediction = !gcloud ai-platform predict --model=xgb_mortgage --region='global' --json-
instances=predictions.json --version=$VERSION_NAME --verbosity=none

print(prediction)
```

輸出內容應會顯示模型的預測結果。這個範例已獲准，因此您應該會看到接近 1 的值。

## 7. 使用 What-If Tool 解讀模型

### 步驟 1：建立假設情境工具的視覺化效果

如要將 [What-if Tool](#) 連線至 AI Platform 模型，您需要將測試範例的子集連同這些範例的基準真相值傳遞至該工具。讓我們建立 500 個測試範例的 NumPy 陣列，以及這些範例的實際資料標籤：

**注意：**您可以變更 `num_wit_examples`，在小工具中載入更多或更少的範例。

```
num_wit_examples = 500
test_examples =
np.hstack((x_test[:num_wit_examples].values,y_test[:num_wit_examples].reshape(-1,1)))
```

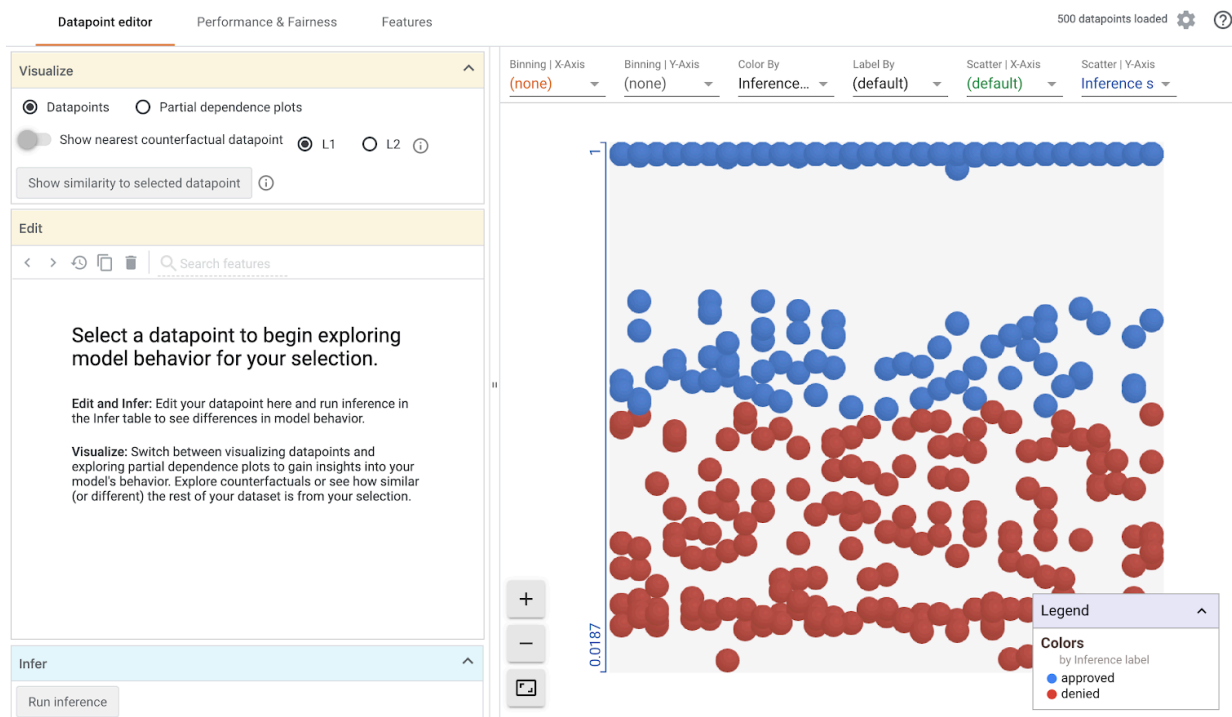
如要例項化 What-If Tool，只要建立 `WitConfigBuilder` 物件，並將要分析的 AI Platform 模型傳遞給該物件即可。

我們在此使用選用的 `adjust_prediction` 參數，因為 What-if Tool 預期模型中的每個類別 (本例為 2 個) 都有一份分數清單。由於模型只會傳回 0 到 1 的單一值，因此我們會在函式中將其轉換為正確格式：

```
def adjust_prediction(pred):
    return [1 - pred, pred]

config_builder = (WitConfigBuilder(test_examples.tolist(), data.columns.tolist() +
['mortgage_status'])
    .set_ai_platform_model(GCP_PROJECT, MODEL_NAME, VERSION_NAME,
adjust_prediction=adjust_prediction)
    .set_target_feature('mortgage_status')
    .set_label_vocab(['denied', 'approved']))
WitWidget(config_builder, height=800)
```

請注意，系統需要一分鐘才能載入圖表。載入後，您應該會看到下列內容：



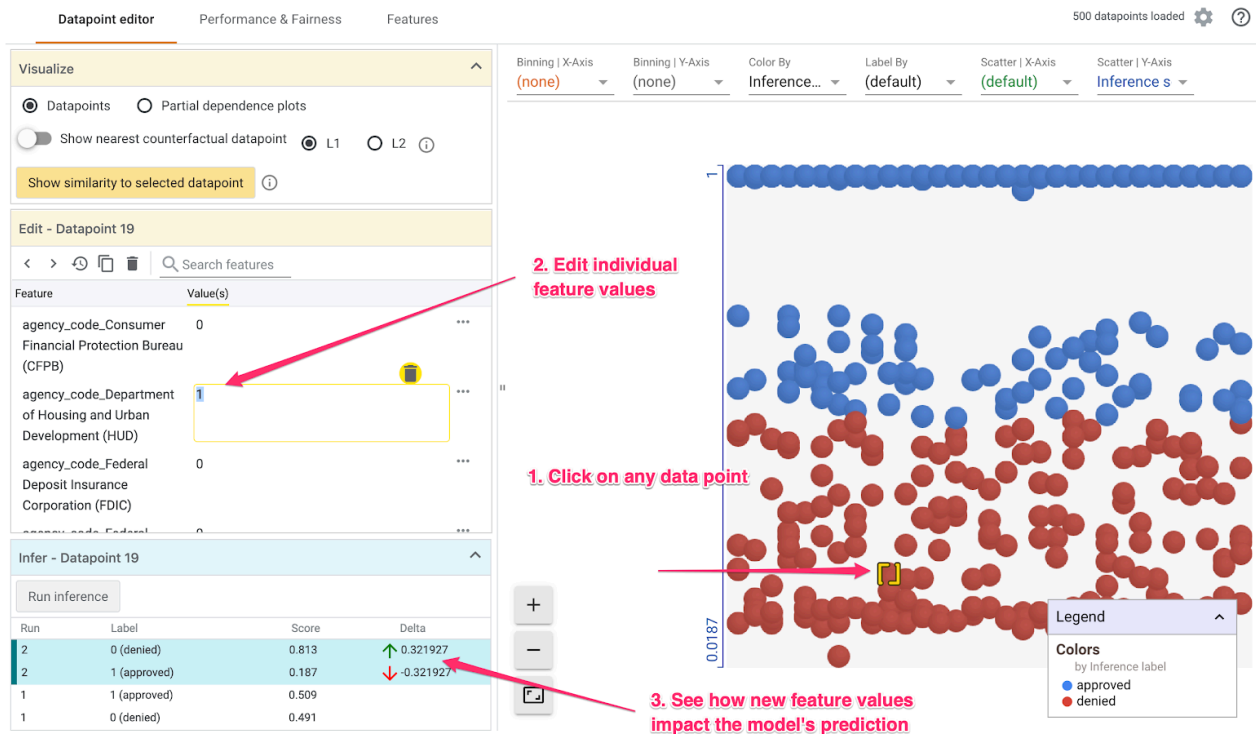
y 軸顯示模型的預測結果，其中 1 代表高可信度 approved 預測，0 則代表高可信度 denied 預測。X 軸只是所有載入資料點的分布範圍。

**注意：**由於您訓練的是自己的模型，且所有機器學習模型都與權重和偏差初始化相關聯，因此您的視覺化內容可能與這裡顯示的略有不同。

## 步驟 2：探索個別資料點

What-If Tool 的預設檢視畫面是「資料點編輯器」分頁。您可以在這裡點選任何個別資料點，查看其特徵、變更特徵值，並瞭解這項變更對模型預測個別資料點的影響。

在下方範例中，我們選擇的資料點接近 .5 門檻。與這個特定資料點相關聯的房貸申請來自 CFPB。我們將該特徵變更為 0，並將 `agency_code` Department of Housing and Urban Development (HUD) 的值變更為 1，藉此瞭解如果這筆貸款改由 HUD 發放，模型的預測結果會如何變化：



如「假設情境工具」左下方的部分所示，變更這項特徵後，模型的 **approved** 預測值大幅減少了 32%。這可能表示貸款來源機構對模型輸出內容有重大影響，但我們需要進行更多分析才能確定。

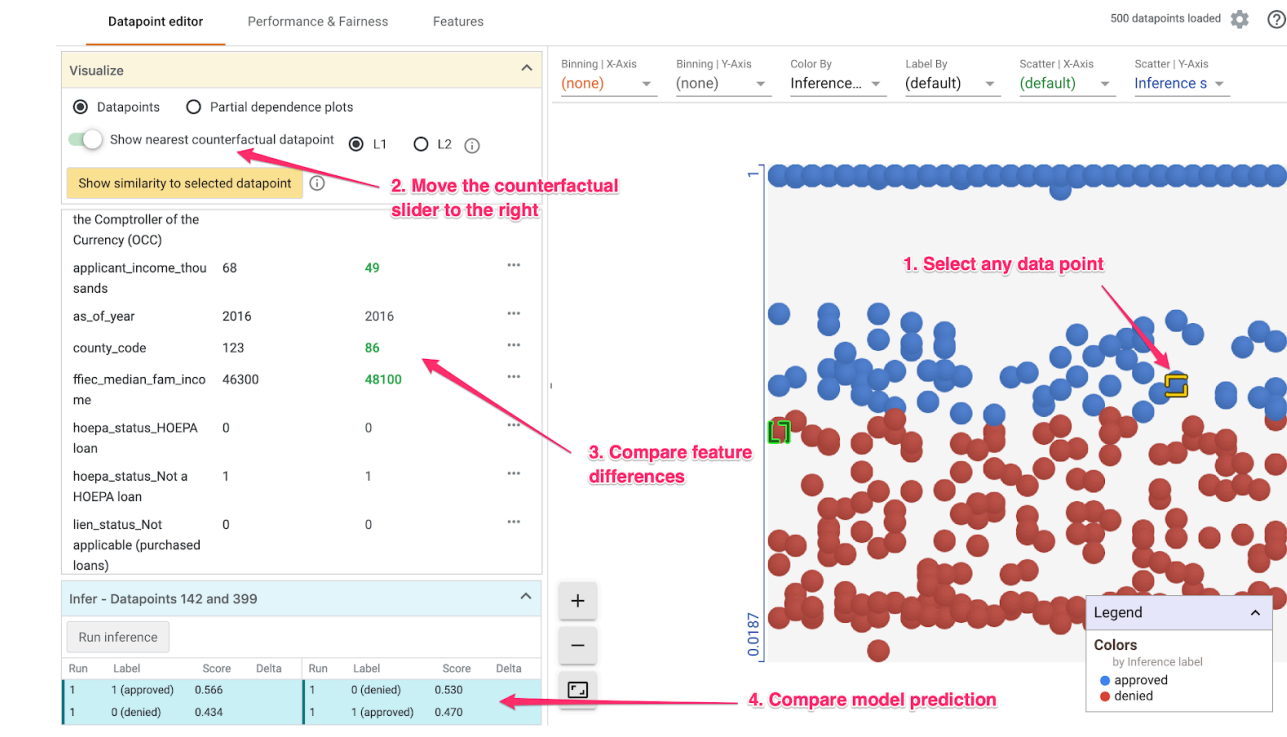
在使用者介面的左下部分，我們也可以看到每個資料點的實際值，並與模型的預測值進行比較：

| Actual value |              | Model's prediction |       |
|--------------|--------------|--------------------|-------|
| Run          | Label        | Score              | Delta |
| 1            | 1 (approved) | 0.658              |       |
| 1            | 0 (denied)   | 0.342              |       |

### 步驟 3：反事實分析

接著，點選任一資料點，然後將「顯示最接近的反事實資料點」滑桿向右移動：

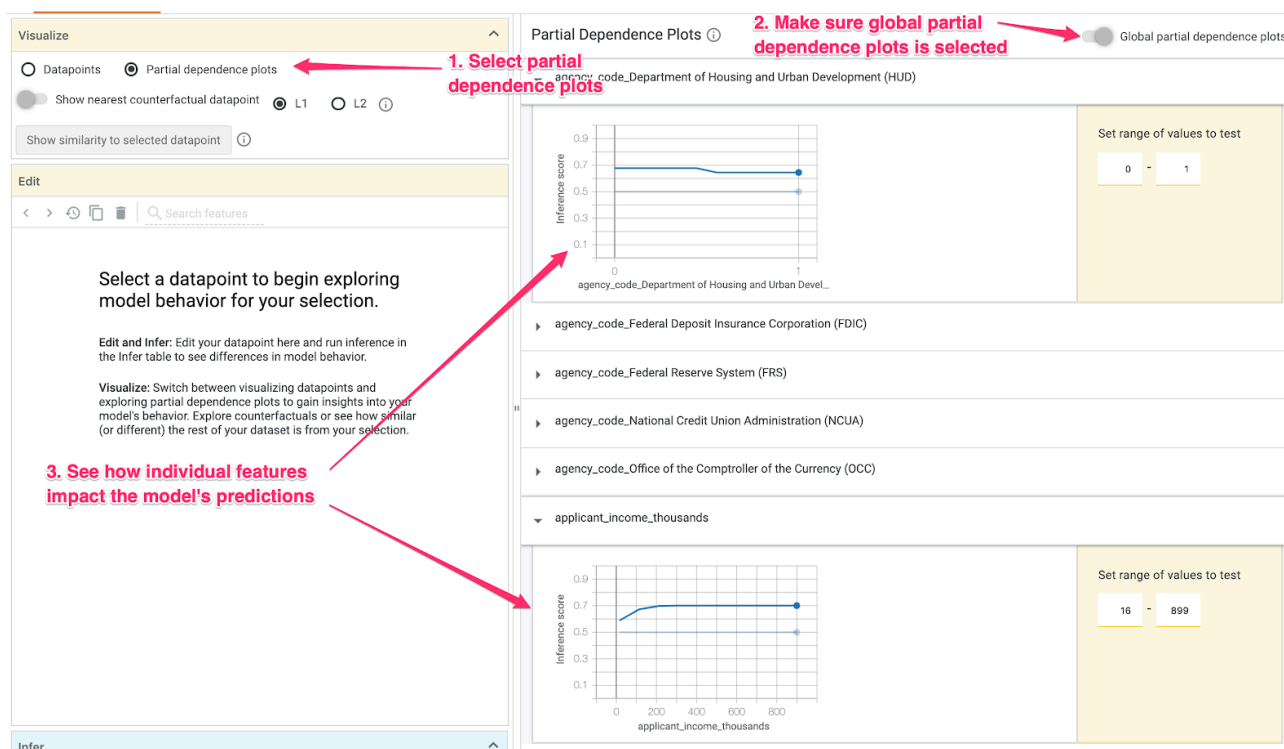




選取這個選項後，系統會顯示與您選取的原始資料點特徵值最相似，但預測結果相反的資料點。接著捲動瀏覽特徵值，即可查看兩個資料點的差異 (差異會以綠色粗體字醒目顯示)。

## 步驟 4：查看偏依賴關係圖

如要查看各項特徵對模型整體預測的影響，請勾選「偏依圖」方塊，並確認已選取「全域偏依圖」：



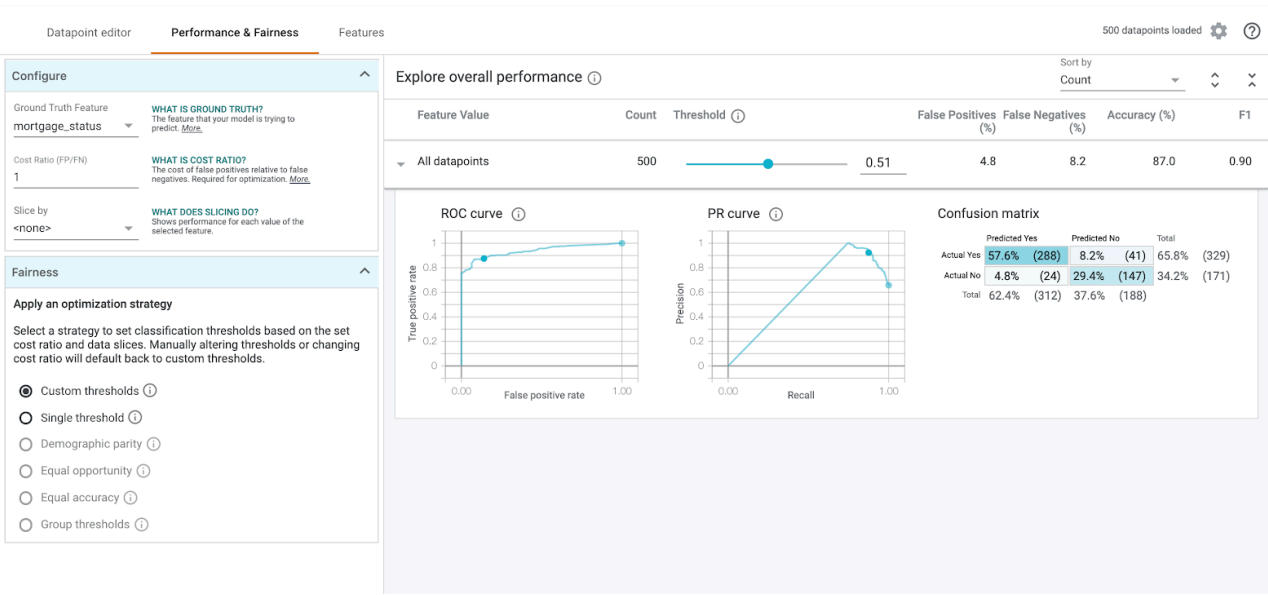
從這裡可以看出，HUD 貸款遭拒的機率略高。由於機構代碼是布林特徵，因此值只能是 0 或 1，圖表才會呈現這種形狀。

`applicant_income_thousands` 是數值特徵，從偏依程度圖中可以看出，收入越高，申請獲得核准的機率就越高，但收入達到約 \$20 萬美元後，機率就不會再增加。超過 \$20 萬美元後，這項特徵就不會影響模型的預測結果。

## 步驟 5：探索整體成效和公平性

接著前往「成效與公平性」分頁。這會顯示模型在所提供資料集上的整體成效統計資料，包括混淆矩陣、PR 曲線和 ROC 曲線。

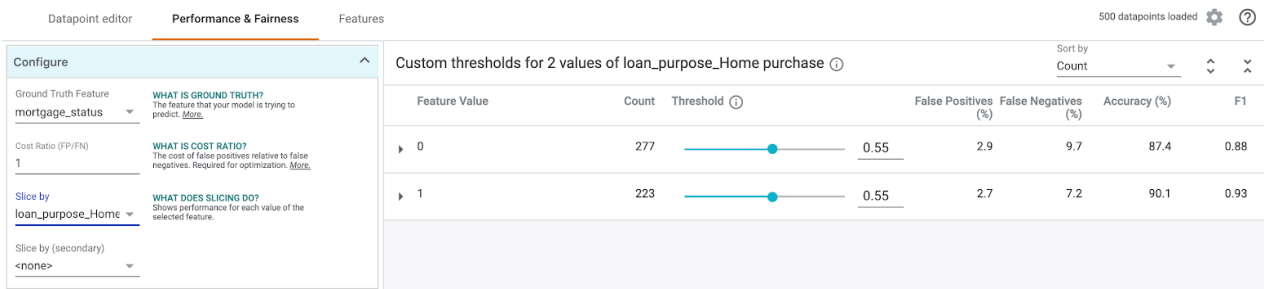
選取「實際資料特徵」 `mortgage_status`，即可查看混淆矩陣：



這個混淆矩陣會以總數百分比的形式，顯示模型的正確和錯誤預測結果。如果將「實際為 Yes / 預測為 Yes」和「實際為 No / 預測為 No」方塊加總，應該會與模型的準確度相同 (約 87%)。

您也可以實驗門檻滑桿，提高和降低模型需要傳回的正向分類分數，然後再決定是否預測貸款為 `approved`，並查看這項操作如何改變準確率、偽陽性和偽陰性的結果。在本例中，門檻約為 **0.55** 時，準確度最高。

接著，在左側的「依據切片」下拉式選單中，選取 `loan_purpose_Home_purchase`：



現在您會看到兩組資料的成效：「0」區塊代表貸款並非用於購屋，「1」區塊則代表貸款用於購屋。比較這兩個區隔的準確度、偽陽率和偽陰性率，找出成效差異。

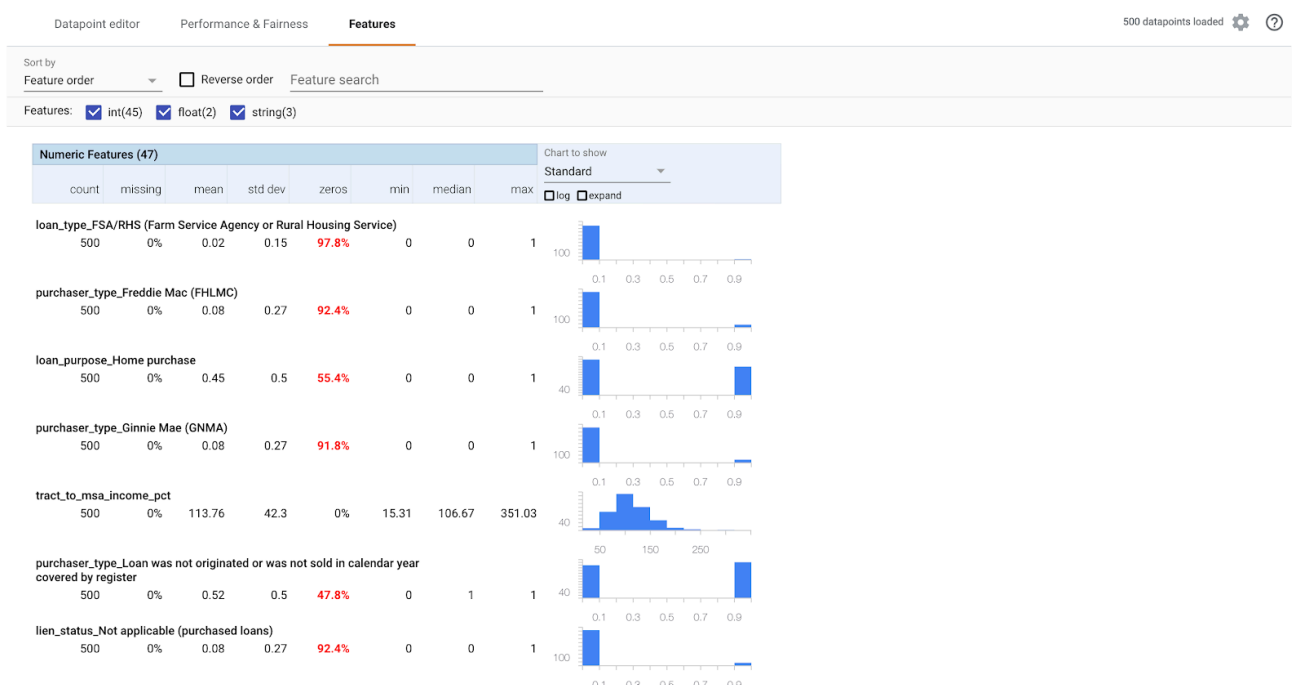
展開資料列查看混淆矩陣，您會發現模型預測的「核准」貸款申請中，有 70% 是用於購屋，只有 46% 是用於其他用途 (確切百分比會因模型而異)：



如果從左側的圓形按鈕選取「群體均等」，系統會調整這兩個門檻，讓模型預測兩個切片中類似百分比的申請人會獲得 approved。這會對每個切片的準確度、偽陽性和偽陰性造成什麼影響？

## 步驟 6：探索功能發布情形

最後，前往 What-if Tool 的「特徵」分頁。這會顯示資料集中每個特徵的值分布情形：



您可以使用這個分頁，確保資料集平衡。舉例來說，資料集中來自農場服務署的貸款似乎很少。為提高模型準確率，我們可能會考慮加入該機構的更多貸款 (如有資料)。

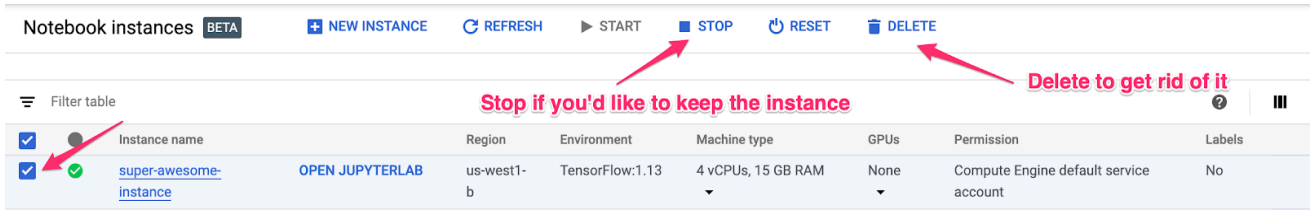
我們在此僅說明幾項假設情境工具的探索構想。歡迎繼續使用這項工具，還有許多領域值得探索！

---

步驟 8 · 約 1 分鐘

## 8. 清除所用資源

如要繼續使用這部筆電，建議在不使用時關機。在 Cloud 控制台的 Notebooks 使用者介面中，選取筆記本，然後選取「停止」：



如要刪除在本實驗室中建立的所有資源，請直接刪除筆記本執行個體，而不是停止執行個體。

使用 Cloud 控制台的導覽選單瀏覽至「儲存空間」，然後刪除您建立的兩個 bucket，以儲存模型資產。